

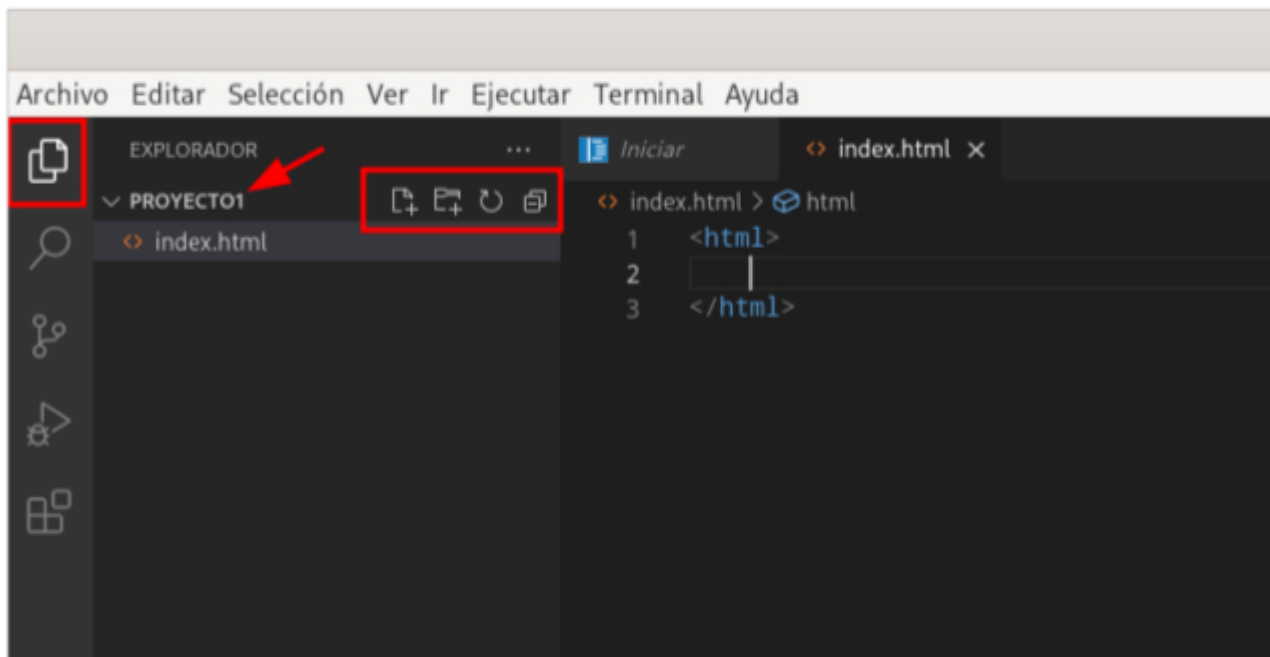
3. Entornos de desarrollo

3.1. Editor

El editor recomendado es **Visual Studio Code**, un editor de **código abierto** que ofrece compatibilidad con JavaScript, HTML y CSS (entre muchos otros lenguajes) y proporciona multitud de **plugins** para facilitar el desarrollo de aplicaciones web.

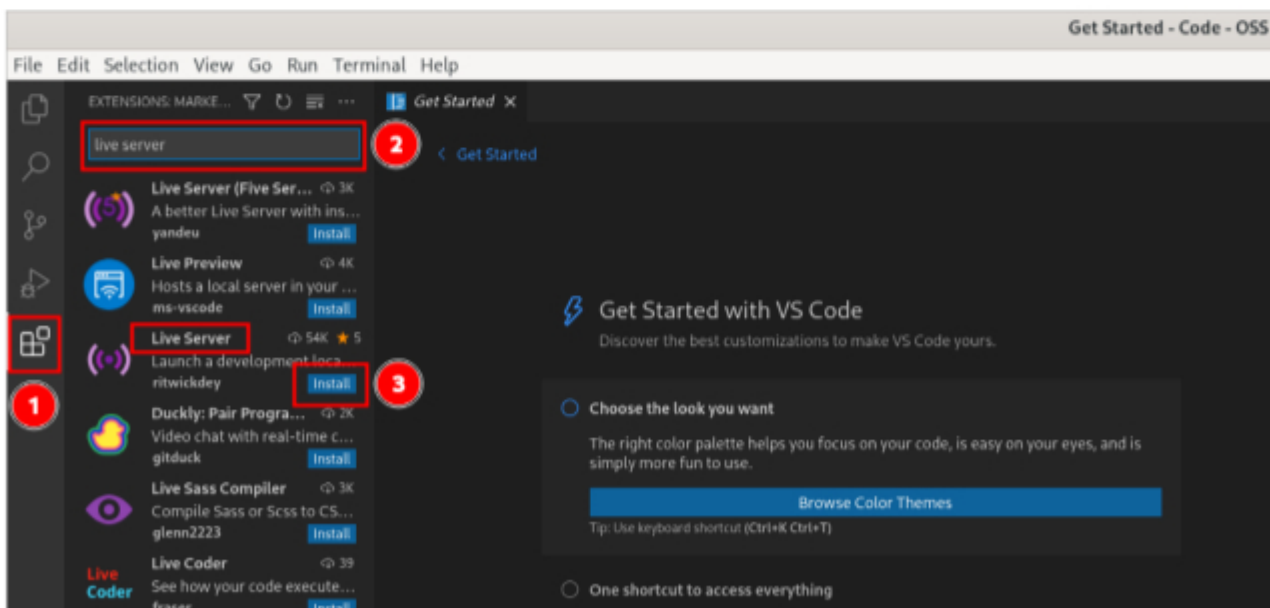
La descarga del editor puede realizarse desde su sitio web oficial, que puedes consultar en los enlaces de la unidad.

Una de las características más interesantes del editor es que permite **abrir una carpeta** como espacio de trabajo del proyecto. Una vez seleccionada la carpeta de trabajo, podremos navegar por su contenido y crear nuevos archivos y carpetas de manera sencilla.



Creación de carpetas.

También dispone de infinidad de **plugins** para facilitar la edición de código. La instalación de *plugins* se realiza de manera muy sencilla, pulsando en el icono correspondiente:



Plugins.



Algunos de los *plugins* más interesantes que usaremos son:

- **Spanish Language Pack.** Paquete de traducción de la interfaz para idioma español.
- **Emmet.** Es una extensión especialmente útil para **editar** **ficheros HTML**. Utiliza **sintaxis CSS** para generar etiquetas HTML de manera rápida y sencilla. Viene instalado por defecto en Visual Studio Code, aunque puede instalarse en cualquier editor adicional. Por ejemplo, si escribimos **div.clase1** y a continuación pulsamos la tecla **Tabulador**, se generará el siguiente código HTML:

```
<div class="clase1"></div>
```

Puede utilizarse cualquier **selector CSS**.

- **Live Server.** Es una extensión que sirve para **iniciar un servidor web** en la carpeta de trabajo. Al instalarla, aparece un icono en la parte inferior derecha del editor. Al pulsar en dicho icono se ejecuta un servidor web en el equipo local, accesible a través de un determinado puerto (normalmente, el puerto 5500): al acceder a la dirección de dicho servidor (normalmente, **http://localhost:5500/**) se accede al contenido de la carpeta de trabajo. Es muy útil cuando se necesita acceder a una página web a través de un servidor, ya que algunas de las funciones de JavaScript no funcionan correctamente si se accede a la página a través del sistema de ficheros convencional (haciendo doble clic sobre el archivo HTML directamente). Así, si tenemos un archivo llamado **pagina1.html** dentro de una carpeta llamada **sitio1**, dentro de la carpeta de trabajo, se podrá acceder a dicha página a través de la ruta **http://localhost:5500/sitio1/pagina1.html**. Pulsando de nuevo sobre el icono, se cerrará el servidor.



Live Server.



IMPORTANTE

Podemos saber si estamos cargando una página en el navegador a través del sistema de ficheros si su URL comienza por **file:///**.

3.2. NodeJS

NodeJS es un **entorno de ejecución** de JavaScript. Nos permite ejecutar código JavaScript **sin necesidad de utilizar un navegador**. De esta manera, podemos usar JavaScript como un lenguaje de programación genérico para diseñar todo tipo de aplicaciones y servicios, independientemente del navegador.

Es importante dejar claro que NodeJS es solo el motor de JavaScript. Por tanto, **no dispone de acceso** a las **API del navegador** (funciones y objetos JavaScript definidos por defecto en el navegador). Por ello, las aplicaciones JavaScript que necesiten acceder a las API del navegador **no funcionarán correctamente en NodeJS**. Algunos ejemplos de API del navegador son los siguientes:

- Objeto **Window**. Hace referencia a la ventana del navegador. La popular función **window.alert** o simplemente **alert** es un ejemplo de código que no funciona en NodeJS. Todas las funciones que hagan referencia al DOM también entran en esta categoría.
- API AJAX. Son las llamadas a funciones para realizar peticiones asíncronas.
- API para manipular gráficos. Se utilizan para crear gráficos vectoriales.
- API de dispositivos. Se utilizan para acceder a las capacidades de los dispositivos: web-cams, vibración, GPS, etc.
- API de almacenamiento local. Se utilizan para almacenar información en los navegadores.

! IMPORTANTE

Es posible utilizar librerías de NodeJS para emular y acceder a algunas de dichas funcionalidades o sus equivalentes.

Al instalar NodeJS también se instala el gestor de paquetes **NPM** (Node Package Manager). Es una herramienta de línea de comandos que permite instalar paquetes de software que se descargan de un repositorio centralizado en Internet. Muchos de estos paquetes se utilizan en el desarrollo web en entorno cliente, ya que proporcionan utilidades o funciones que amplían o mejoran las proporcionadas por los navegadores; otras se utilizan en el desarrollo de aplicaciones de escritorio o en el desarrollo en entorno servidor; por último, muchas otras son **utilidades**, como herramientas para crear esqueletos de aplicaciones que utilizan un determinado *framework* (Angular, React, VueJS), herramientas de automatización para la realización de **test** o herramientas para **desplegar aplicaciones**.

Los paquetes NPM se pueden instalar de **tres maneras**:

- A nivel de sistema: los ficheros del paquete se instalan en las carpetas del sistema operativo.
- A nivel de usuario: los ficheros del paquete se instalan en la carpeta de usuario.
- A nivel local: los ficheros del paquete se instalan en la **carpeta del proyecto**.

! IMPORTANTE

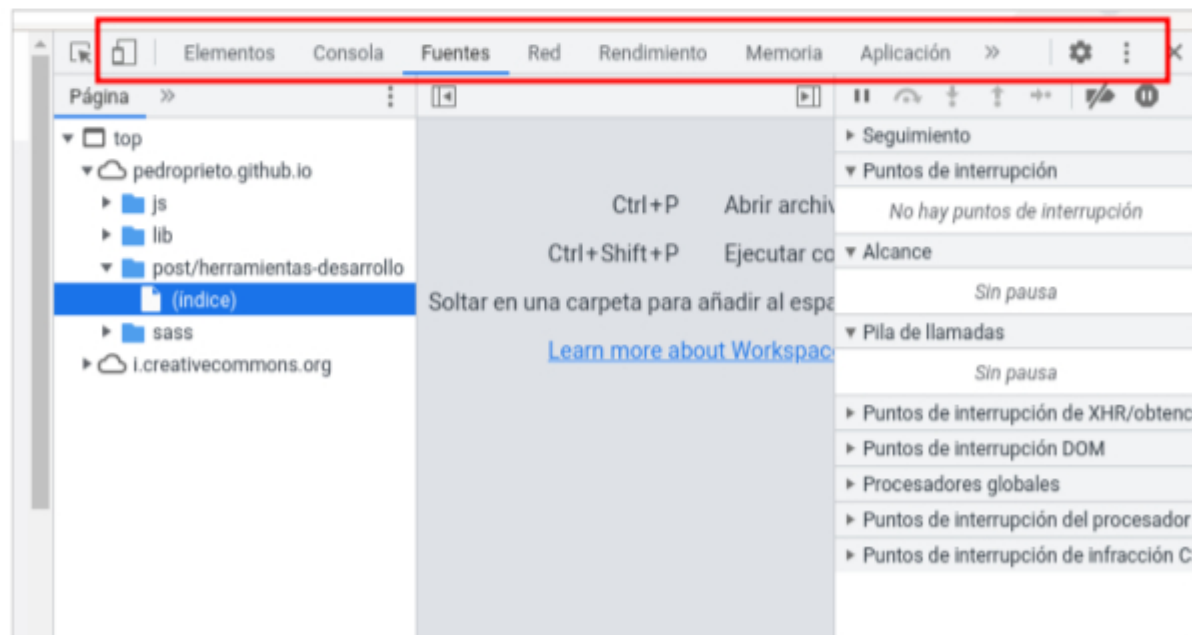
Normalmente, los paquetes de sistema los instala el administrador del equipo, mientras que los paquetes locales o de usuario pueden ser instalados por cualquier usuario.

Los paquetes instalados a nivel de sistema o de usuario suelen ser programas que se pueden ejecutar (normalmente a través de la línea de comando); por ejemplo, herramientas para crear proyectos nuevos.

Los paquetes locales suelen ser librerías de las que depende el proyecto que se está desarrollando: no suelen ser ejecutables.

3.3. Herramientas de desarrollo del navegador

Las herramientas de desarrollo del navegador son accesibles pulsando la tecla **F12**, o **Alt+Cmd+I** en macOS.





Estas herramientas son fundamentales para comprobar el funcionamiento del código creado y depurar errores. Algunas de las funcionalidades más importantes que ofrecen son:

Pestañas	
Fuentes	Permite ver los archivos que componen la web que se está visualizando. Al acceder a un fichero de JavaScript es posible definir puntos de interrupción (<i>breakpoints</i>) pulsando sobre la línea de código correspondiente. También es posible examinar variables y realizar otras tareas de depuración. Por último, si se utiliza en el código, es posible interactuar con el comando debugger , que interrumpirá la ejecución en el punto indicado si la consola de desarrollo está abierta.
Consola	Muestra los mensajes generados desde el código JavaScript de la página (mediante el comando console.log) y permite ejecutar código JavaScript , incluyendo las funciones exportadas desde el código de la página. Estudiaremos con más detalle el funcionamiento de la consola en el apartado siguiente.
Red	Permite ver las peticiones de recursos que realiza la página. Es muy útil al trabajar con aplicaciones SPA, ya que permite inspeccionar las peticiones que se realizan en segundo plano mediante AJAX.
Inspector (Firefox) o Elementos (Chrome)	Permite ver las propiedades DOM de los elementos HTML. También se accede a ella pulsando con el botón derecho sobre un elemento de la página y seleccionando <i>Inspeccionar</i> .

3.4. REPL

JavaScript ofrece un entorno **REPL** (acrónimo que significa *read-evaluate-print loop* o bucle de lectura, evaluación e impresión). Este entorno no es más que una **consola** que nos permite escribir cualquier código JavaScript como código de entrada y devolver el resultado a la consola después de la ejecución.

El REPL es muy útil a la hora de hacer pruebas rápidas o pequeños prototipos. Podemos acceder al REPL de JavaScript en el **navegador** y en **NodeJS**.

A. EL REPL EN EL NAVEGADOR

El REPL del navegador es la **consola** de las **herramientas de desarrollo**.

Si el navegador tiene abierta una pestaña vacía (sin ninguna web cargada), no habrá ningún código JavaScript cargado; si, por el contrario, no lo está, seguramente haya algún código JavaScript cargado con el que se podría interactuar.

Como ejemplo, puedes escribir el siguiente código en la consola y comprobar su ejecución:

```
var a = 4;
var b = 5;
var c = a + b;
alert(c);
```

Verás que el código se ejecuta y que por pantalla se muestra un cuadro de alerta con el resultado.

También verás que tras ejecutar cada línea (o tras ejecutar la última, si las has copiado juntas) aparece el resultado **undefined**. Esto se debe a que en entornos REPL todo se considera una función que devuelve un valor. Como ninguna de las líneas de nuestro primer programa devuelve valores (**var** es una declaración de variables y **alert** no devuelve ningún valor en la consola, sino que solo muestra el texto en el cuadro de alerta), el resultado de cada ejecución es **undefined**.

Puedes hacer alguna prueba más con algún código sencillo y ver cómo funciona.



B. EL REPL EN NODEJS

Para acceder al REPL en NodeJS hay que abrir una consola en el sistema operativo (**CMD** en Windows o un **terminal** en GNU/Linux o Mac) y ejecutar el comando **node**. Si lo has instalado correctamente, verás que aparece un intérprete de comandos de JavaScript.

Para salir de la consola de Node puedes ejecutar **Ctrl+D** o pulsar **Ctrl+C** dos veces. También puedes escribir **.help** para obtener información.

Como ejemplo, puedes escribir el siguiente código en la consola y comprobar su ejecución:

```
var a = 4;
var b = 5;
var c = a + b;
alert(c);
```

Verás que en este caso las tres primeras líneas se ejecutan sin problemas, pero la llamada a la función **alert** produce un error. Esto es debido a lo que hemos comentado antes sobre las API web: NodeJS solo es intérprete de JavaScript y no tiene acceso a las API de navegador, entre las que se encuentra la función **window.alert**.

Por el contrario, si ejecutas el siguiente código, verás que funciona correctamente:

```
var a = 4;
var b = 5;
var c = a + b;
console.log(c);
```

En este último caso, el programa no hace referencia a ninguna API de navegador.

Como en el caso del navegador, puedes hacer alguna prueba más con algún código sencillo y ver cómo funciona. Comprueba también que puedes utilizar teclas de **autocompletar** de manera similar a como se usan en la consola de GNU/Linux.

3.5. Servicios de validación

La mayoría de los editores incorporan herramientas para comprobar la sintaxis del código. Aun así, en ocasiones es necesario recurrir a servicios de validación para buscar o corregir errores. Algunos de los más populares son:

- JSLint para [JavaScript](#).
- The W3C Markup Validation Service para [HTML](#).
- The W3C CSS Validation Service para [CSS](#).

3.6. Otras herramientas

A continuación, se enumeran otras herramientas muy utilizadas en el desarrollo web en entorno cliente:

- **Control de versiones.** La herramienta más utilizada, con diferencia, es **git**. Suele utilizarse junto con plataformas como **GitHub**, **GitLab** o **BitBucket** para publicar el código y colaborar con otros desarrolladores.
- **Pruebas y test.** En el desarrollo de aplicaciones, es fundamental realizar pruebas de código. En el caso del desarrollo en entorno cliente, se utilizan **herramientas y librerías de NPM** que se ejecutan sobre **NodeJS** o el propio navegador. Los test tienen especial importancia en el **desarrollo guiado por pruebas** (*test-driven development*, o TDD), una metodología de desarrollo que consiste en:
 1. Escribir los test (pruebas unitarias) antes que el código.
 2. Crear el código necesario para superar dichos test.
 3. Refactorizar el código escrito para eliminar duplicidades y hacerlo más limpio y eficiente.

- **Integración continua.** Es una metodología que combina el control de versiones y las pruebas, para hacer integraciones automáticas de manera frecuente. Permite a los desarrolladores realizar comprobaciones automáticas de los cambios que realicen en el código mediante test e **incorporar esos cambios a la rama principal de desarrollo** si se superan dichos test. Algunas de las herramientas que se utilizan para ello son **GitHub Actions** o **Travis CI**.
- **Despliegue o entrega continua.** Va un paso más allá de la integración continua, añadiendo el **despliegue o publicación** de la aplicación si el nuevo código supera la fase de test.